



CertusPro-NX Defect Detection on SOM Board Demo User Guide

User Guide

FPGA-UG-02248-1.1

February 2026

Disclaimers

Lattice makes no warranty, representation, or guarantee regarding the accuracy of information contained in this document or the suitability of its products for any particular purpose. All information herein is provided AS IS, with all faults, and all associated risk is the responsibility entirely of the Buyer. The information provided herein is for informational purposes only and may contain technical inaccuracies or omissions, and may be otherwise rendered inaccurate for many reasons, and Lattice assumes no obligation to update or otherwise correct or revise this information. Products sold by Lattice have been subject to limited testing and it is the Buyer's responsibility to independently determine the suitability of any products and to test and verify the same. LATTICE PRODUCTS AND SERVICES ARE NOT DESIGNED, MANUFACTURED, OR TESTED FOR USE IN LIFE OR SAFETY CRITICAL SYSTEMS, HAZARDOUS ENVIRONMENTS, OR ANY OTHER ENVIRONMENTS REQUIRING FAIL-SAFE PERFORMANCE, INCLUDING ANY APPLICATION IN WHICH THE FAILURE OF THE PRODUCT OR SERVICE COULD LEAD TO DEATH, PERSONAL INJURY, SEVERE PROPERTY DAMAGE OR ENVIRONMENTAL HARM (COLLECTIVELY, "HIGH-RISK USES"). FURTHER, BUYER MUST TAKE PRUDENT STEPS TO PROTECT AGAINST PRODUCT AND SERVICE FAILURES, INCLUDING PROVIDING APPROPRIATE REDUNDANCIES, FAIL-SAFE FEATURES, AND/OR SHUT-DOWN MECHANISMS. LATTICE EXPRESSLY DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY OF FITNESS OF THE PRODUCTS OR SERVICES FOR HIGH-RISK USES. The information provided in this document is proprietary to Lattice Semiconductor, and Lattice reserves the right to make any changes to the information in this document or to any products at any time without notice.

Inclusive Language

This document was created consistent with Lattice Semiconductor's inclusive language policy. In some cases, the language in underlying tools and other items may not yet have been updated. Please refer to Lattice's inclusive language [FAQ 6878](#) for a cross reference of terms. Note in some cases such as register names and state names it has been necessary to continue to utilize older terminology for compatibility.

Contents

Contents	3
Abbreviations in This Document.....	5
1. Introduction	6
2. Functional Description.....	7
3. Demo Setup	8
3.1. Hardware Requirement.....	8
3.2. Software Requirements	11
4. Programming the Demo	12
4.1. Setting Up Raspberry Pi CM5	12
4.1.1. Preparing Hardware for Imaging	12
4.1.2. Installing rpiboot on Windows PC	13
4.1.3. Installing Raspberry Pi Imager on Windows PC.....	13
4.1.4. Imaging Raspberry Pi CM5	13
4.1.5. Configuring Raspberry Pi CM5 Network.....	14
4.2. Updating HUB Package.....	15
4.3. Updating Software Package	16
4.4. Programming MachXO3D FPGA Using Radiant Programmer.....	16
4.5. Programming CertusPro-NX SOM with DD Demo Using Radiant Programmer	16
5. Running the Demo	19
5.1. Preparing the Device	19
5.2. Starting Scripts in VNC	19
5.2.1. Connecting RealVNC Viewer to IP Address	19
5.2.2. Running Scripts.....	19
5.3. Using HUB	20
5.4. Preparing Hardware Setup	21
5.4.1. Printing Dataset.....	21
5.4.2. Focusing Camera	21
5.4.3. Adjusting Light Brightness	21
5.5. Training from Dataset	21
5.5.1. Capturing Datasets	21
5.6. Training Model	23
5.7. Compiling and Programming for FPGA	23
5.7.1. Compile RISC-V Binary.....	23
5.7.2. Compile Defect Detection Network	23
5.7.3. Prepare Binary File with Reference Vector and Threshold	23
5.7.4. Prepare Binary Image	24
5.7.5. Flash Prepared Binary Image.....	24
5.8. Running Trained Model.....	24
References	27
Technical Support Assistance	28
Revision History.....	29

Figures

Figure 3.1. CertusPro-NX SOM Board	8
Figure 3.2. CertusPro-NX Carrier Module	9
Figure 3.3. Raspberry Pi CM5	9
Figure 3.4. IMX219 Camera Module	9
Figure 3.5. Camera Focus Tool.....	10
Figure 3.6. Bottom of SOM and Top of Carrier Board with Connection Details	10
Figure 3.7. SOM-Carrier-Raspberry Pi Assembly	10
Figure 3.8. SOM-Carrier-Raspberry Pi Assembly with USB-C Power and Ethernet Connected for Normal Operation	11
Figure 4.1. Demo Package Folder Structure	12
Figure 4.2. Jumper Location on the Carrier Module at J10	12
Figure 4.3. Raspberry Pi CM5 to Windows PC Connection for Imaging	13
Figure 4.4. rpiboot.exe Successfully Sets the Raspberry Pi CM5 to Mass Storage	14
Figure 4.5. Windows Terminal Accessing SOM through Ethernet.....	15
Figure 4.6. CertusPro-NX (LFPCPX-100) Device Listed in Radiant Programmer Software	16
Figure 4.7. Radiant Programmer SPI Flash Option for Binary File Programming	17
Figure 4.8. Programming FPGA from Radiant Programmer Software.....	17
Figure 4.9. Successful Operation Output.....	18
Figure 4.10. Radiant Programmer SPI Flash Options for Binary File Programming	18
Figure 5.1. Terminal Output from HUB when a Client is Connected	19
Figure 5.2. A Network Trained on Vitamins Matching a Non-Defective Sample	20
Figure 5.3. A Network Trained on Vitamins Matching a Defective Sample	20
Figure 5.4. Adjust J24 Jumpers According to the Pattern for UART Mode	22
Figure 5.5. Adjust J24 Jumpers According to the Pattern for I2C Mode.....	22
Figure 5.6. Threshold Value	23
Figure 5.7. Top View of Camera with Selected Object of Trained Model.....	25
Figure 5.8. Profile View of Camera with Selected Object of Trained Model	25
Figure 5.9. Defect Detected in Selected Object of Trained Model	26

Abbreviations in This Document

A list of abbreviations used in this document.

Abbreviation	Definition
AC/DC	Alternating Current/Direct Current
AI	Artificial Intelligence
CM5	Compute Module Five
DD	Defect Detection
ESD	Electrostatic Discharge
FPGA	Field-Programmable Gate Array
HUB	Host-accelerated Unified Bridge
I2C	Inter-Integrated Circuit
IP Address	Internet Protocol Address
ML	Machine Learning
OPN	Ordering Part Number
OS	Operating System
PC	Personal Computer
RAM	Random Access Memory
RISC-V	Reduced Instruction Set Computer Five
SDK	Software Development Kit
SSH	Secure Shell
SOM	System On Module
SOP	System On Platform
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
VNC	Virtual Network Computing

1. Introduction

Defect detection (DD) plays a crucial role in enabling machines to distinguish between normal objects and those with visual defects. In production facilities and assembly lines, the ability to accurately detect defects in real time is fundamental to most manufacturing processes.

2. Functional Description

This document describes DD running on a system-on-module (SOM) hosting a Lattice CertusPro™-NX device. The aim is to showcase the solution's capabilities in identifying and localizing various objects within images or video streams.

A key focus of this implementation is the deployment of DD on small form factor devices, such as edge AI modules, embedded systems, and mobile platforms. These compact yet powerful devices enable real-time inference at the edge, reduce latency, preserve privacy, and minimize the need for cloud-based processing.

3. Demo Setup

3.1. Hardware Requirement

The following hardware components are required to run the DD demo on the CertusPro-NX SOM:

- CertusPro-NX SOM Board:
 - OPN: LFCPNX-SOM-EVN
 - CertusPro-NX Package: LFCPNX-100-9ASG256I
 - MachXO3D™ Package: LCMXO3D-9400
- CertusPro-NX Carrier Module (LF-GEN-CR-PCBA)
- Raspberry Pi Compute Module 5 (CM5)
- USB-A/USB-C to USB-C cable (Used for data connection between the Raspberry Pi CM5 and a host PC)
- USB-A to micro-USB cable (Used for programming the bitstream, firmware, and ensuring proper terminal prints)
- IMX219 Camera Module: Raspberry Pi Camera Module 2/Arducam 8 MP IMX219 Camera with four-lane support.
- Power adapter for board power: 5 V 25 W AC/DC external wall mount (class II) adapter multi-blade (sold separately) input.
- Camera focus tool
- USB-powered ring light or light box with 5,000 K – 5,800 K white light
- Ethernet cable
- Camera holder

Refer to the following figures for the views of each hardware module and the complete board assembly.

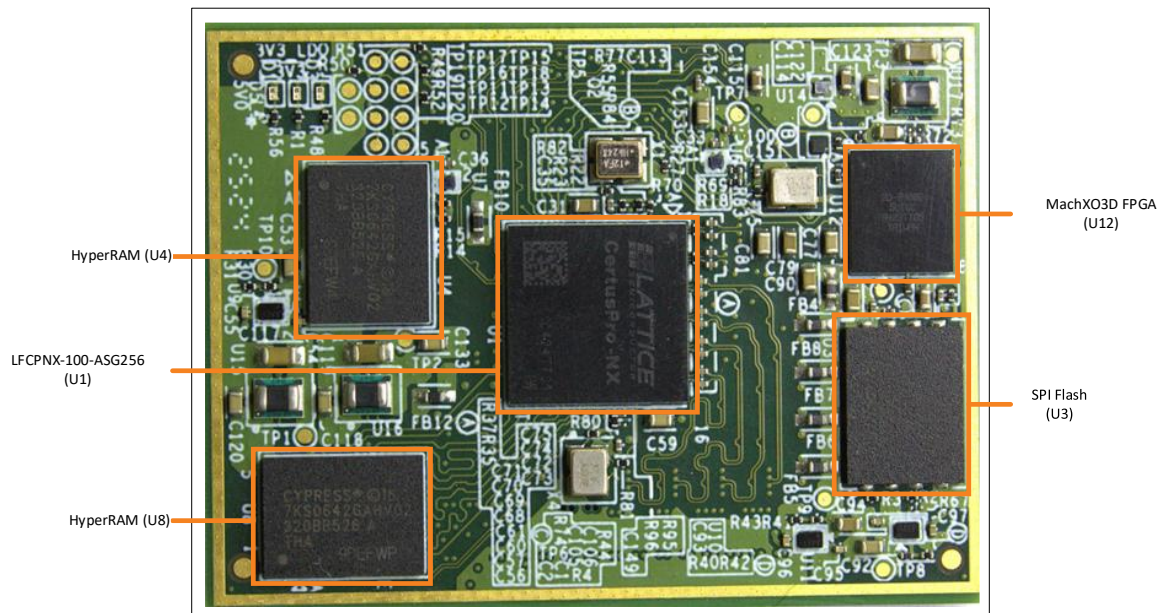


Figure 3.1. CertusPro-NX SOM Board

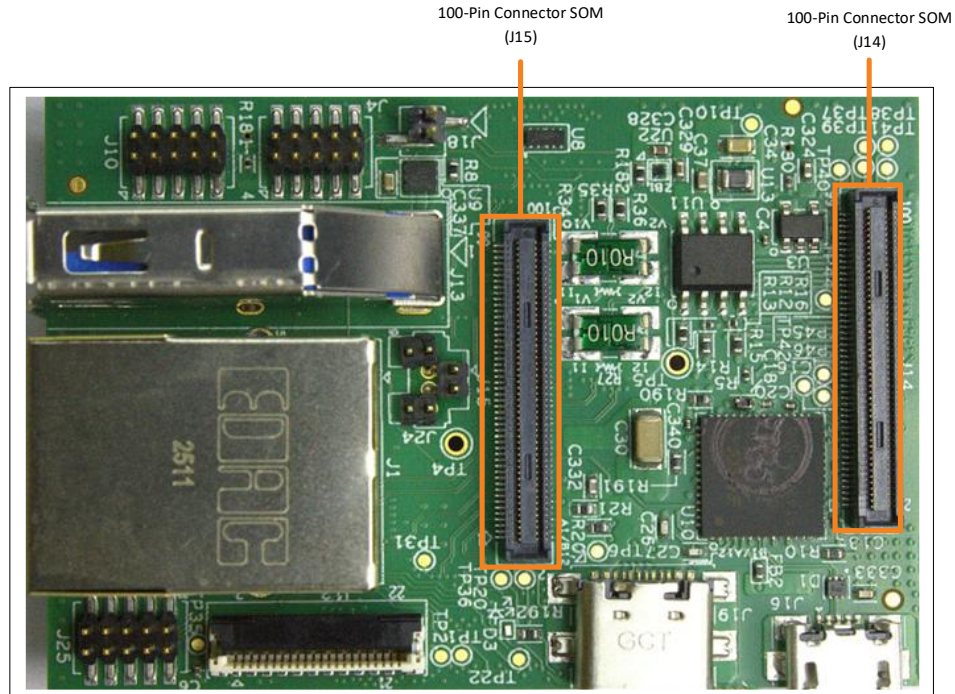


Figure 3.2. CertusPro-NX Carrier Module

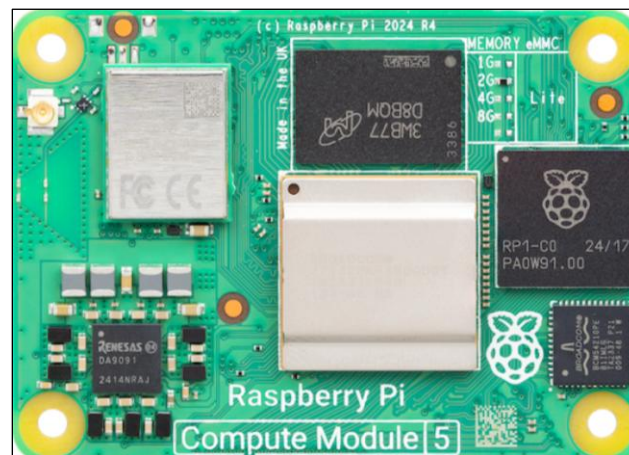


Figure 3.3. Raspberry Pi CM5



Figure 3.4. IMX219 Camera Module



Figure 3.5. Camera Focus Tool

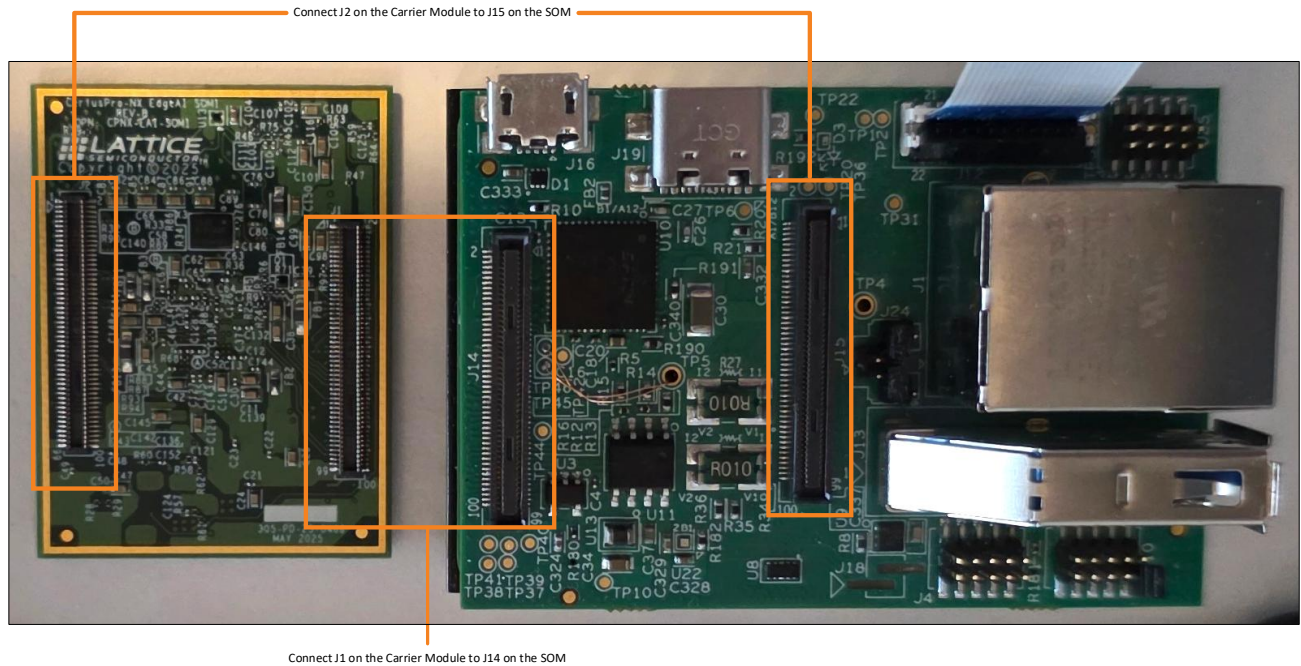


Figure 3.6. Bottom of SOM and Top of Carrier Board with Connection Details

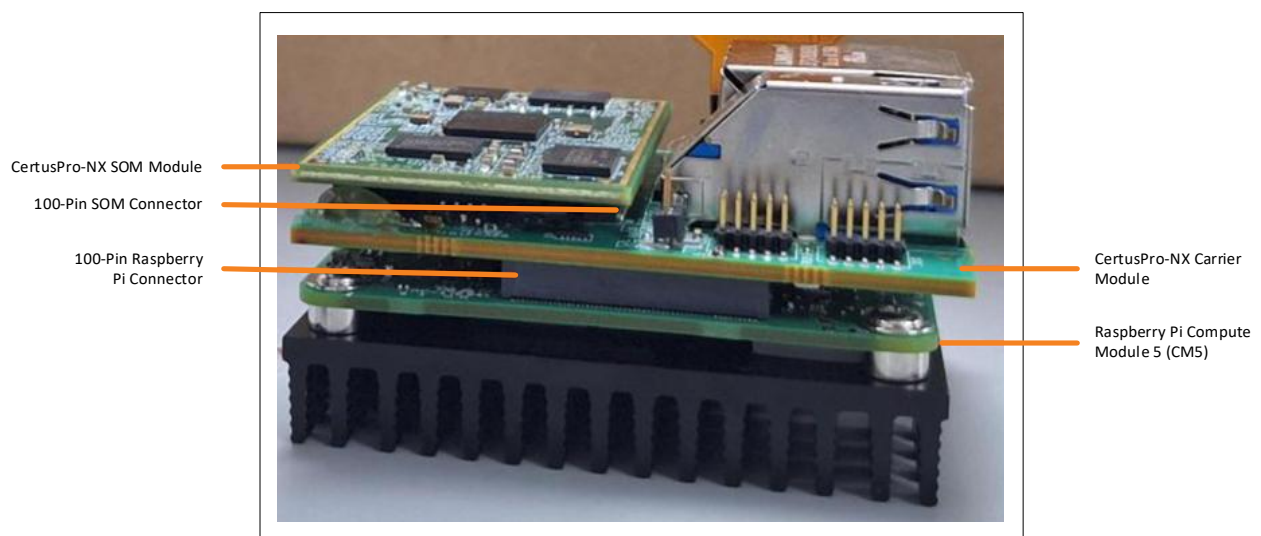


Figure 3.7. SOM-Carrier-Raspberry Pi Assembly

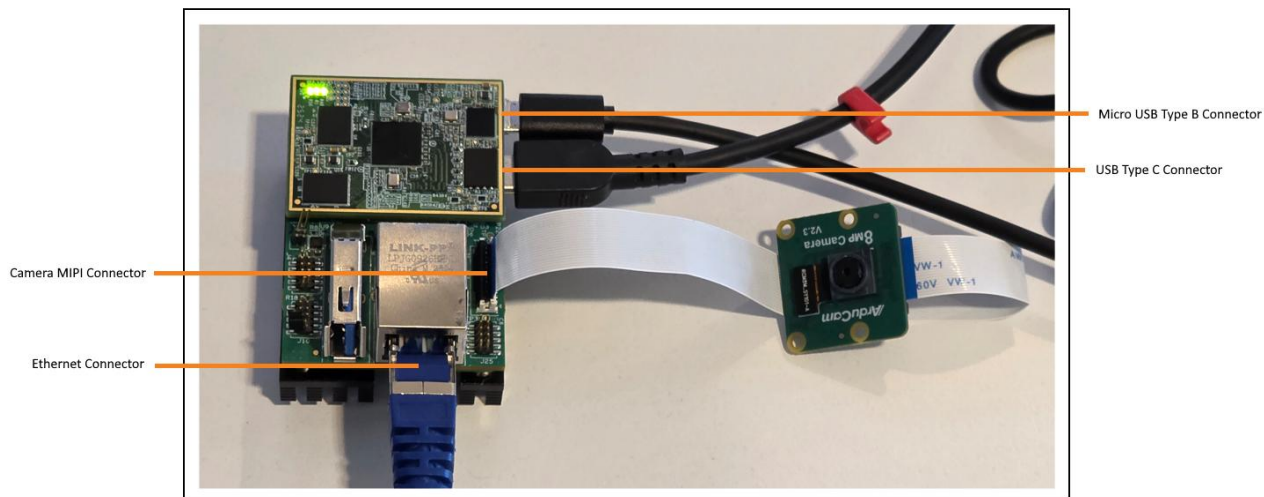


Figure 3.8. SOM-Carrier-Raspberry Pi Assembly with USB-C Power and Ethernet Connected for Normal Operation

3.2. Software Requirements

The following software tools and files are required to run the DD demo on the CertusPro-NX SOM:

- [Lattice Radiant™ Programmer](#) (version 2025.1 or later).
- [Raspberry Pi Software](#)
- [rpiboot](#)
- [Demo package](#) containing:
 - Raspberry Pi OS Image: <date>-CPNX-HUB-1.5-edgeHUB-<version>.img.xz
 - Bitstream: mod_top_torna_revb_cpnx100_impl_1-<version>.bit @ 0x00000000
 - FwRoot: fwroot_<version>.bin @ 0x00300000
 - Workbench
 - sensAI SDK (included in sensAI 8.0 or later)
 - Lattice Propel (2025.1 or later)
 - DefectDetectionTestPatternsA.pdf

4. Programming the Demo

This section focuses on programming the components of the SOM to run the DD demo.

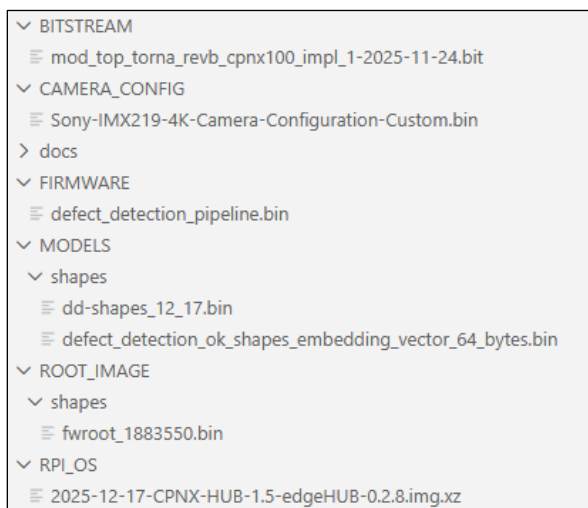


Figure 4.1. Demo Package Folder Structure

4.1. Setting Up Raspberry Pi CM5

This section describes how to download and image the Raspberry Pi CM5 operating system with preinstalled software and configurations.

4.1.1. Preparing Hardware for Imaging

Warning: The SOM assembly and associated circuitry are delicate high-speed equipment and highly susceptible to ESD. Ensure that you use proper ESD protection measures before handling.

As shown in the figure below, on the SOM assembly, add a 1.27 mm jumper on pins 1 and 2 (Jumper J10).

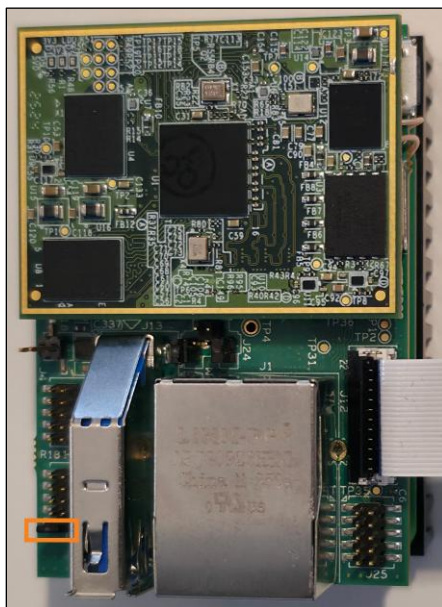


Figure 4.2. Jumper Location on the Carrier Module at J10

4.1.2. Installing rpiboot on Windows PC

rpiboot installs the drivers and boot tool. Do not close any driver installation windows that appear during the installation process.

By default, this package installs at the *C:\Program Files (x86)\Raspberry Pi* directory.

Follow these steps:

1. Install the provided **rpiboot_setup.exe**.
2. Double-click **rpiboot.exe** to run it.

4.1.3. Installing Raspberry Pi Imager on Windows PC

1. Install the provided **imager-1.9.6.exe**.
2. Double-click the installer to run it.
3. Select and agree with the policies, then continue with the installation.

4.1.4. Imaging Raspberry Pi CM5

1. Connect the USB-C cable from the device to a Windows PC (use the USB-C port next to the micro-USB port).
Note: Ensure that the PC is able to provide enough power for imaging the Raspberry Pi CM5. Do not power the device from a PC during normal operation.

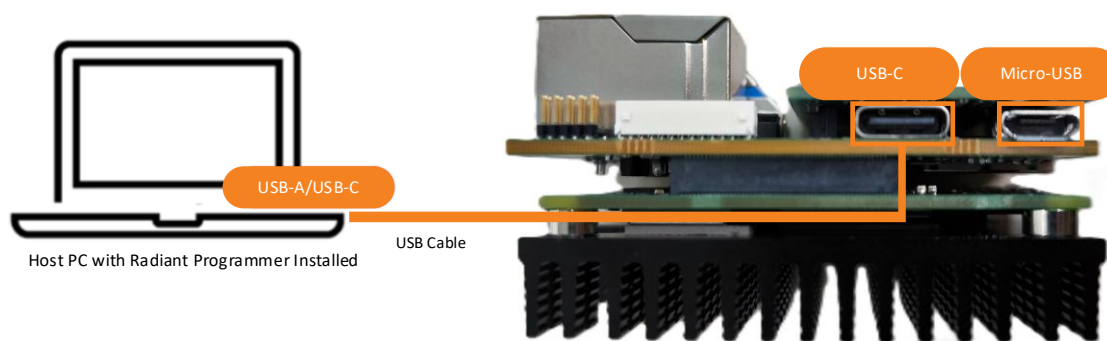
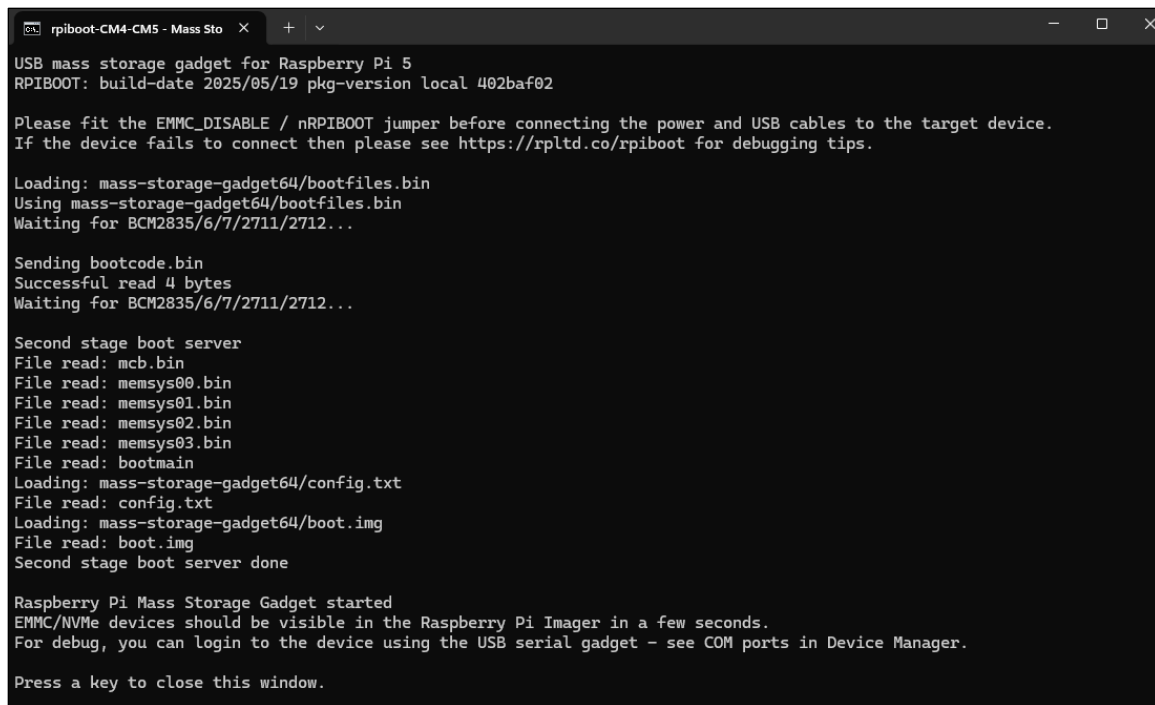


Figure 4.3. Raspberry Pi CM5 to Windows PC Connection for Imaging

2. From Windows, run rpiboot:
`C:\Program Files (x86)\Raspberry Pi\rpiboot.exe`

The Raspberry Pi CM5 is detected by the script, and a storage device appears in Explorer.



```
rpiboot-CM4-CM5 - Mass Storage
USB mass storage gadget for Raspberry Pi 5
RPiBOOT: build-date 2025/05/19 pkg-version local 402baf02

Please fit the EMMC_DISABLE / nRPiBOOT jumper before connecting the power and USB cables to the target device.
If the device fails to connect then please see https://rpltd.co/rpiboot for debugging tips.

Loading: mass-storage-gadget64/bootfiles.bin
Using mass-storage-gadget64/bootfiles.bin
Waiting for BCM2835/6/7/2711/2712...

Sending bootcode.bin
Successful read 4 bytes
Waiting for BCM2835/6/7/2711/2712...

Second stage boot server
File read: mcb.bin
File read: memsys00.bin
File read: memsys01.bin
File read: memsys02.bin
File read: memsys03.bin
File read: bootmain
Loading: mass-storage-gadget64/config.txt
File read: config.txt
Loading: mass-storage-gadget64/boot.img
File read: boot.img
Second stage boot server done

Raspberry Pi Mass Storage Gadget started
EMMC/NVMe devices should be visible in the Raspberry Pi Imager in a few seconds.
For debug, you can login to the device using the USB serial gadget - see COM ports in Device Manager.

Press a key to close this window.
```

Figure 4.4. rpiboot.exe Successfully Sets the Raspberry Pi CM5 to Mass Storage

3. Open Raspberry Pi Imager:
 - a. Click **Choose Device**, then select the Raspberry Pi CM5 option.
 - b. Click **Choose OS**, then select the *.img* file from the demo package.
 - c. Click **Choose Storage**, then select the Raspberry Pi CM5 storage device (if no other storage devices are attached, it shows as the only available one).
 - d. When prompted for **Customization**, select **No**.
 - e. Click **Next** or **WRITE** to write the image to the storage.
 - f. Once complete, a pop-up window indicates that the process is complete and the device can be safely disconnected.
4. Remove the USB-C cable and the jumper.
5. Connect the USB-C power adapter.

4.1.5. Configuring Raspberry Pi CM5 Network

The HUB Raspberry Pi OS image comes pre-installed with an ISC DHCP Server for serving dynamic IP addresses to devices connected to the Ethernet port (*eth0*). The DHCP server starts automatically on boot with a static IP address.

To set up and use the Ethernet connection for the SOM Raspberry Pi CM5 after flashing:

- Your machine should have an Ethernet port with dynamic IP address configured.
- Connect an Ethernet LAN cable from your machine to the SOM Ethernet port.
- Connect the power supply to the SOM.
- Optionally, as described in the previous section, use PuTTY to access the serial debug UART and confirm the IP address.

A static IP address *10.10.1.1/24* is automatically assigned to the SOM Raspberry Pi CM5. If this IP address is pre-allotted, a subsequent IP in the range *10.10.1.100* to *10.10.1.200* is assigned.

Run the following command to check the IP address:

```
hostname -I
```

The IP address is now accessible within the network.

In case of failure, power cycle the SOM for a seamless reboot and IP allocation.

The following figure shows an example of accessing the SOM assembly over SSH. Use the default credentials to log in.

Default credentials:

- **Username:** *lattice*
- **Password:** *lattice*

```

lattice@lattice-hub: ~
PS C:\> ping 10.10.1.1

Pinging 10.10.1.1 with 32 bytes of data:
Reply from 10.10.1.1: bytes=32 time<1ms TTL=64
Reply from 10.10.1.1: bytes=32 time<1ms TTL=64
Reply from 10.10.1.1: bytes=32 time=2ms TTL=64

Ping statistics for 10.10.1.1:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 2ms, Average = 0ms
Control-C
PS C:\> ssh lattice@10.10.1.1
The authenticity of host '10.10.1.1 (10.10.1.1)' can't be established.
ED25519 key fingerprint is SHA256:4BHi0w0bPJ0WLv5228ztdmqcbwWCU4RvmvL09pj/4JA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.1.1' (ED25519) to the list of known hosts.
Linux lattice-hub 6.12.47+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.12.47-1+rpt1~bookworm (2025-09-16) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Dec 4 22:32:26 2025 from 10.10.1.100
lattice@lattice-hub:~ $

```

Figure 4.5. Windows Terminal Accessing SOM through Ethernet

4.1.5.1. Enabling Static IPv4 Address

This configuration enables the SOM to be accessed over the LAN network. To set a static IP for the Raspberry Pi CM5 on the SOM assembly, run the following command. This needs to be configured only once and will automatically reconnect if available:

```
nmcli connection modify <connection-name> ifname eth0 type ethernet ipv4.addresses <ip-addr>/24 \
ipv4.gateway <gateway> ipv4.method manual autoconnect yes
```

Where:

- **connection-name** – *Wired connection 1* (already set up on Raspberry Pi CM5)
- **ip-addr** – IPv4 address
- **gateway** – IPv4 gateway

Note: Root privileges are required. If you encounter a *permission denied* error, run the command using *sudo* or with a user account that has elevated privileges.

4.2. Updating HUB Package

If a new package is provided without a full Raspberry Pi OS update:

1. Remote into the Raspberry Pi CM5.
2. Copy the *lsc_hub-X.X.X-arm64.deb* package to the Downloads folder on the Raspberry Pi CM5 using a USB key.
3. Install the package in Terminal:
 - a. Open Terminal.
 - b. Run:

```
sudo apt install lsc-hub_X.X.X_arm64.deb -y
```

(Example: lsc-hub-1.5.0-arm64.deb)

4.3. Updating Software Package

If a new package is provided without a full Raspberry Pi OS update:

1. Uninstall the previous package in Terminal:

```
sudo apt remove lsc-hub -y
```

2. Install the latest package in Terminal:

```
sudo apt install lsc-hub_X.X.X_arm64.deb -y
```

(Example: lsc-hub-1.5.0-arm64.deb)

4.4. Programming MachXO3D FPGA Using Radiant Programmer

If the SOM is not detected by the Radiant Programmer tool as an LFCPNX-100 device, you likely need to program the MachXO3D FPGA. For detailed programming instructions, refer to the RTL User Guide available on the [sensAI Reference Design GitHub](#) page.

4.5. Programming CertusPro-NX SOM with DD Demo Using Radiant Programmer

1. Install the Radiant Programmer software in its default location: `C:\lsc\programmer\radiant\2025.1`.
2. Run the Radiant Programmer software:
 - a. Click **Check Devices** to ensure the SOM is detected as an LFCPNX-100 device. If it is not detected, verify the connection or replace the micro-USB cable. Otherwise, refer to the [Programming MachXO3D FPGA Using Radiant Programmer](#) section.
 - b. Enable **LFCPNX** as the listed device.

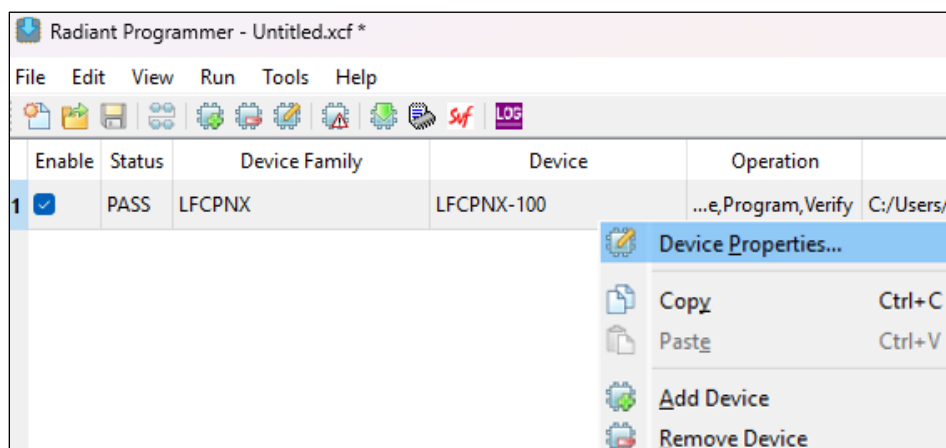


Figure 4.6. CertusPro-NX (LFCPNX-100) Device Listed in Radiant Programmer Software

- c. Right-click the device and select **Device Properties**.

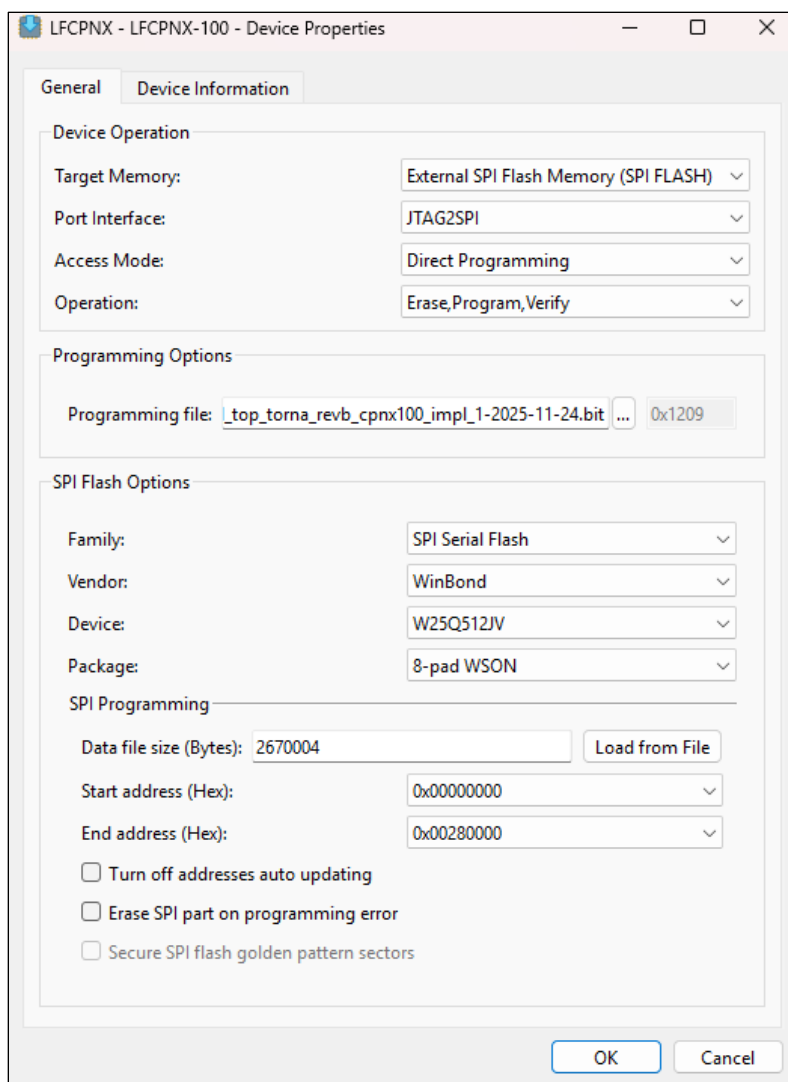


Figure 4.7. Radiant Programmer SPI Flash Option for Binary File Programming

- d. For **Target Memory**, select **External SPI Flash**. This expands the options.
- e. For **Programming file**, select the *.bit* file you downloaded (for example, *mod_top_torna_revb_cpnx100_impl_1-2025-11-24 1.bit*).
- f. Click the Load from File button next to Data file size.
- g. For the **Start address (Hex)**, select *0x00000000*.
- h. Click **OK**.
- i. From the toolbar menu, click **Run**, then select **Program Device**.

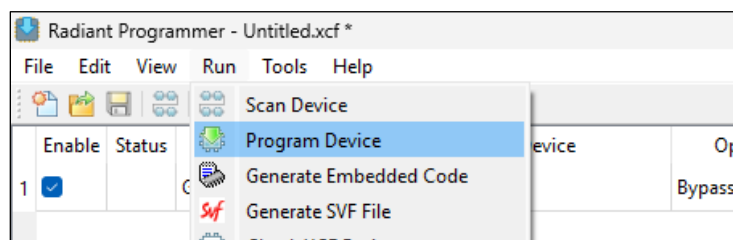


Figure 4.8. Programming FPGA from Radiant Programmer Software

- This process takes approximately 20 minutes. When successful, the following output appears.

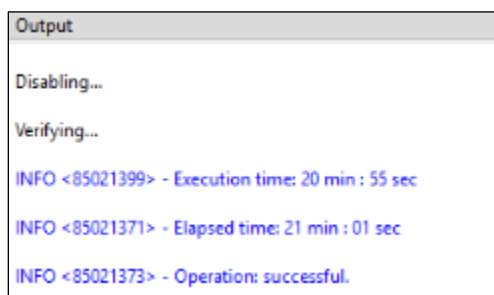


Figure 4.9. Successful Operation Output

- Repeat this process with *FwRoot.bin* and set the start address to *0x00300000*.

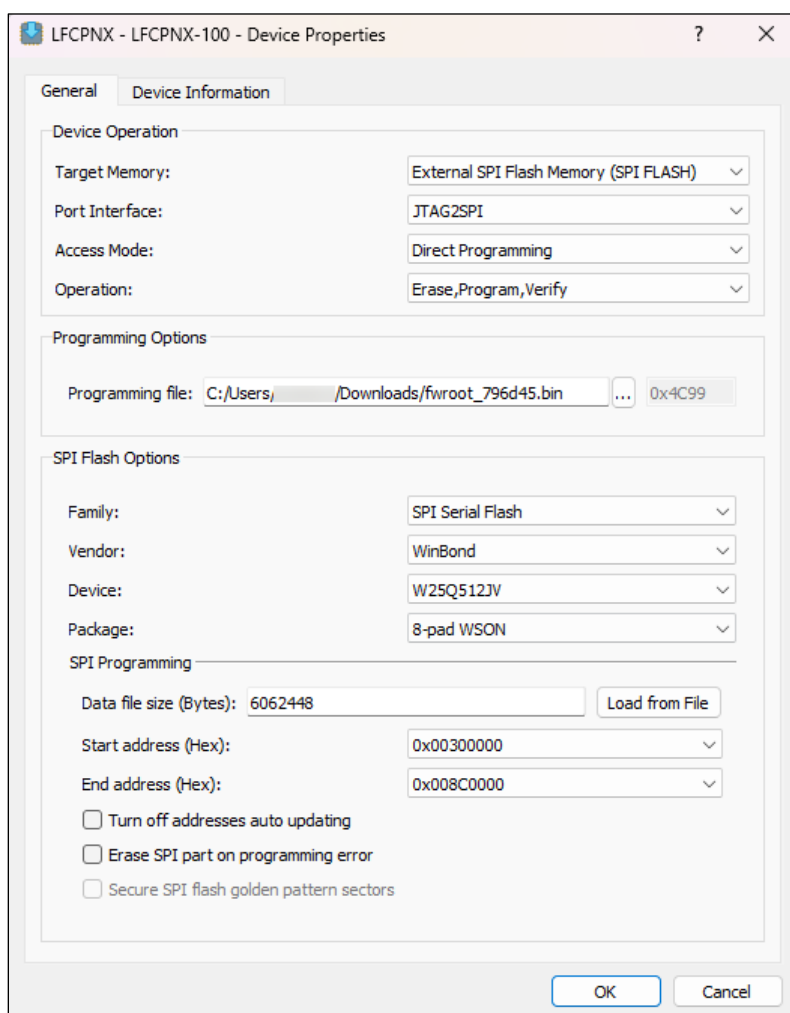


Figure 4.10. Radiant Programmer SPI Flash Options for Binary File Programming

5. Running the Demo

5.1. Preparing the Device

The Raspberry Pi image comes with a static IP address set to `10.10.1.1`. Follow these steps to connect:

1. Connect an Ethernet cable from the SOM to your computer.
2. Connect power to the Raspberry Pi CM5 through the USB-C port next to the micro USB port. If the device is already powered on, power cycle the device by unplugging the power and plugging it back in.

5.2. Starting Scripts in VNC

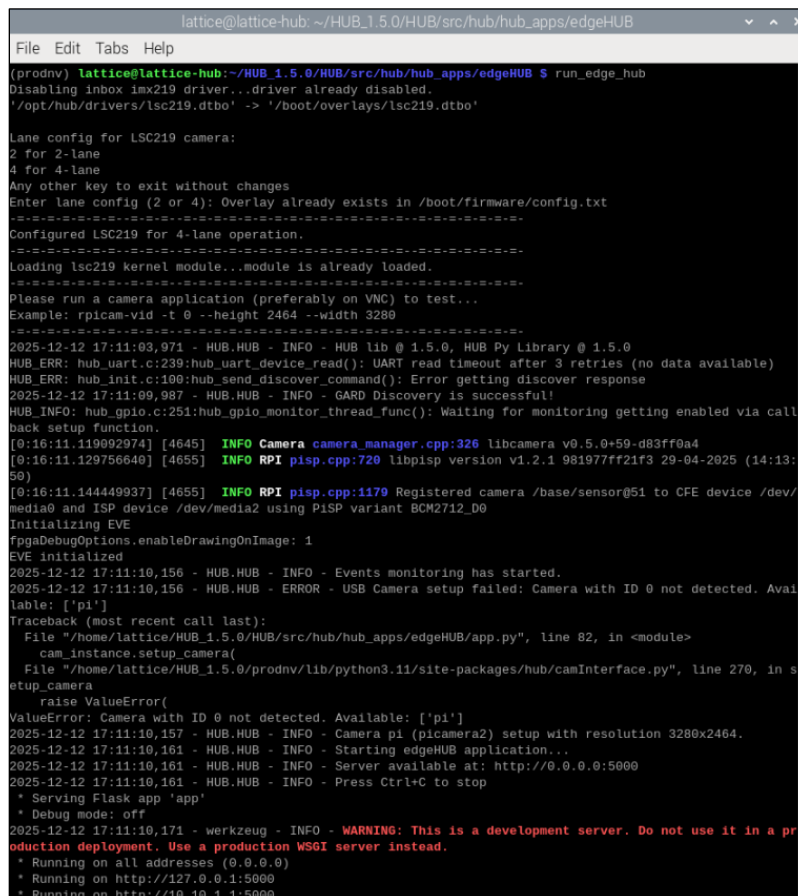
5.2.1. Connecting RealVNC Viewer to IP Address

1. Open RealVNC Viewer.
2. Enter the IP address of the Raspberry Pi CM5 retrieved in the [Preparing the Device](#) section.
3. Enter the username and password.
4. The Raspberry Pi OS desktop appears.

5.2.2. Running Scripts

From the Raspberry Pi OS desktop, open Terminal, and run:

```
run_edge_hub
```



```
lattice@lattice-hub: ~/HUB_1.5.0/HUB/src/hub/hub_apps/edgeHUB
File Edit Tabs Help
(prodenv) lattice@lattice-hub:~/HUB_1.5.0/HUB/src/hub/hub_apps/edgeHUB $ run_edge_hub
Disabling imx219 driver...driver already disabled.
'/opt/hub/drivers/lsc219.dtbo' -> '/boot/overlays/lsc219.dtbo'

Lane config for LSC219 camera:
2 for 2-lane
4 for 4-lane
Any other key to exit without changes
Enter lane config (2 or 4): Overlay already exists in /boot/firmware/config.txt
-----
Configured LSC219 for 4-lane operation.
-----
Loading lsc219 kernel module...module is already loaded.
-----
Please run a camera application (preferably on VNC) to test...
Example: rpicalm-vid -t 0 --height 2464 --width 3280
-----
2025-12-12 17:11:03,971 - HUB.HUB - INFO - HUB lib @ 1.5.0, HUB Py Library @ 1.5.0
HUB_ERR: hub_uart.c:239:hub_uart_device_read(): UART read timeout after 3 retries (no data available)
HUB_ERR: hub_init.c:100:hub_send_discover_command(): Error getting discover response
2025-12-12 17:11:09,987 - HUB.HUB - INFO - GARD Discovery is successful!
HUB_INFO: hub_gpio.c:251:hub_gpio_monitor_thread_func(): Waiting for monitoring getting enabled via call
back setup function.
[0:16:11.119992974] [4645] INFO Camera_manager.cpp:326 libcamera v0.5.0+d83ff0a4
[0:16:11.129756640] [4655] INFO RPI plisp.cpp:720 libplisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:
50)
[0:16:11.144449937] [4655] INFO RPI plisp.cpp:1179 Registered camera /base/sensor@51 to CFE device /dev/
media0 and ISP device /dev/media2 using PISP variant BCM2712_00
Initializing EVE
fpgaDebugOptions.enableDrawingOnImage: 1
EVE initialized
2025-12-12 17:11:10,156 - HUB.HUB - INFO - Events monitoring has started.
2025-12-12 17:11:10,156 - HUB.HUB - ERROR - USB Camera setup failed: Camera with ID 0 not detected. Avail
lable: ['pi']
Traceback (most recent call last):
  File "/home/lattice/HUB_1.5.0/HUB/src/hub/hub_apps/edgeHUB/app.py", line 82, in <module>
    cam_instance.setup_camera(
  File "/home/lattice/HUB_1.5.0/prodenv/lib/python3.11/site-packages/hub/camInterface.py", line 270, in s
etup_camera
    raise ValueError(
ValueError: Camera with ID 0 not detected. Available: ['pi']
2025-12-12 17:11:10,157 - HUB.HUB - INFO - Camera pi (picamera2) setup with resolution 3280x2464.
2025-12-12 17:11:10,161 - HUB.HUB - INFO - Starting edgeHUB application...
2025-12-12 17:11:10,161 - HUB.HUB - INFO - Server available at: http://0.0.0.0:5000
2025-12-12 17:11:10,161 - HUB.HUB - INFO - Press Ctrl+C to stop
* Serving Flask app 'app'
* Debug mode: off
2025-12-12 17:11:10,171 - werkzeug - INFO - WARNING: This is a development server. Do not use it in a pr
oduction deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.10.1.1:5000
```

Figure 5.1. Terminal Output from HUB when a Client is Connected

Note: A few errors may appear because the system attempts to identify available interfaces, such as UART, I2C, and cameras.

5.3. Using HUB

1. Open a web browser on the connected Windows system.
2. Enter the IP address (10.10.1.1) and append the port :5000 in the address bar (for example, 10.10.1.1:5000).
3. Navigate to the **Live Streaming** tab.
4. Click on the **Start Stream** button.

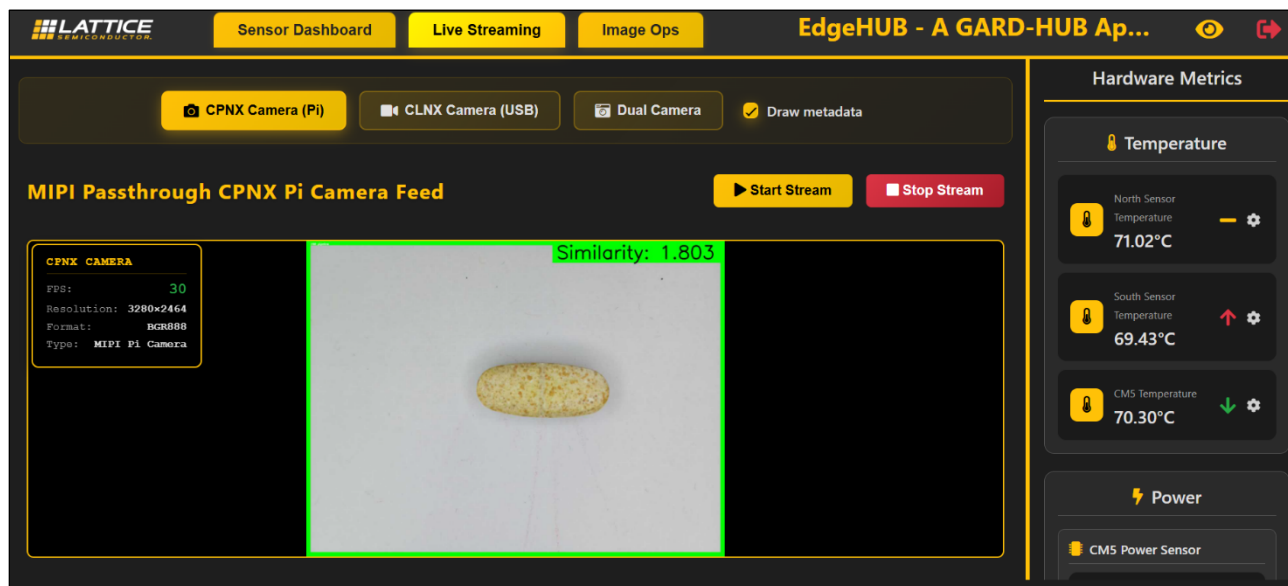


Figure 5.2. A Network Trained on Vitamins Matching a Non-Defective Sample

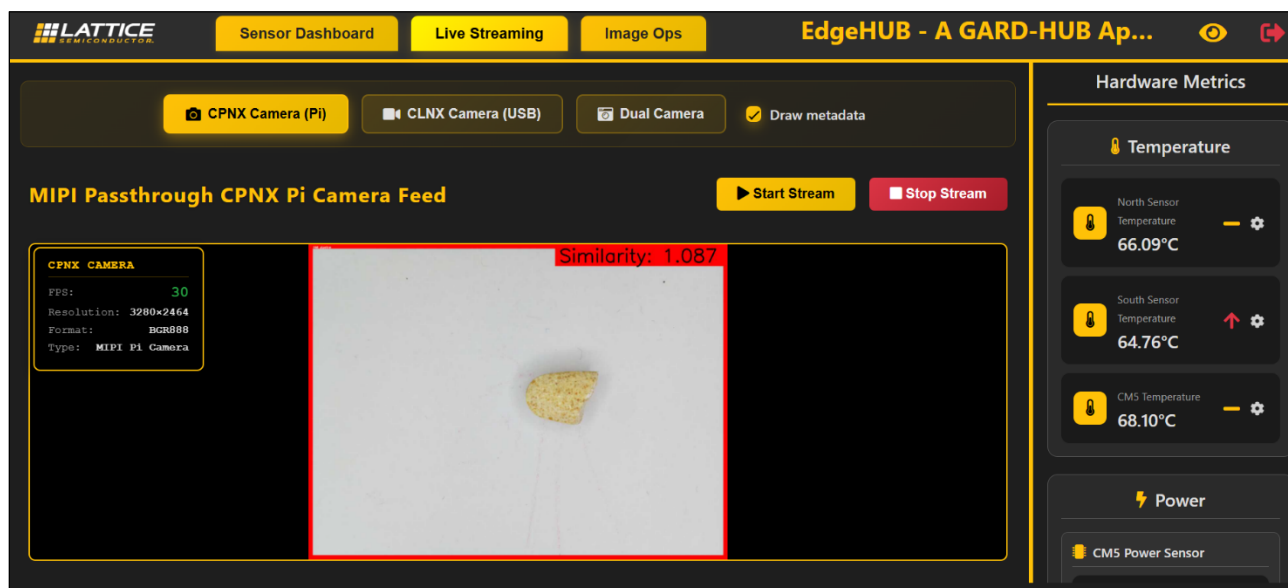


Figure 5.3. A Network Trained on Vitamins Matching a Defective Sample

5.4. Preparing Hardware Setup

5.4.1. Printing Dataset

Print the document *docs/DefectDetectionTestPatternA.pdf* on legal-sized paper using a laser printer.

Ensure that if you use another sheet size, the scale remains the same.

These images are used for the training dataset, validation dataset, and testing dataset.

5.4.2. Focusing Camera

1. Install the camera onto the camera mount by sliding it into the slit with the lens pointing downward.
2. Place the focus page on a flat surface with the camera positioned above it, pointing at the pattern.
3. Adjust the placement by checking the live stream view.
4. Using the camera focus tool, adjust the focus of the camera:
 - Place the tool on the camera lens.
 - Turn clockwise or counterclockwise until the object appears as focused as possible in the live stream view.

5.4.3. Adjusting Light Brightness

1. Place the ring light above the camera so that the camera is in the center of the ring.
2. Ensure that the object is not overexposed or underexposed, and brighten or dim the light brightness as necessary.
3. Navigate to the Capture Tool in EdgeHUB.
4. Click Start Capture. This may take up to 90 seconds to complete.
5. After capturing, the view provided by the FPGA appears. Verify that the object is not overexposed or underexposed. Adjust the light brightness accordingly.
6. Repeat the capture until the object details are clear.

5.5. Training from Dataset

This step is not required if you are running the provided pretrained model.

5.5.1. Capturing Datasets

5.5.1.1. Set Up Device in UART Mode (Optional)

This step is optional but reduces the capture time from 90 seconds in I2C mode to approximately 15 seconds. However, the system must return to I2C mode for any use other than captures.

1. From the Raspberry Pi OS terminal, change the mode of the configuration file:

```
sudo chmod 644 /opt/hub/config/gard_12345.json
```
2. Update the file with the following settings:
 - `"control_bus": "HUB_GARD_BUS_UART"`
 - `"data_bus": "HUB_GARD_BUS_UART"`
3. Power off the system on platform (SOP).
4. Adjust the J24 jumpers on the middle of the carrier board according to the pattern below:

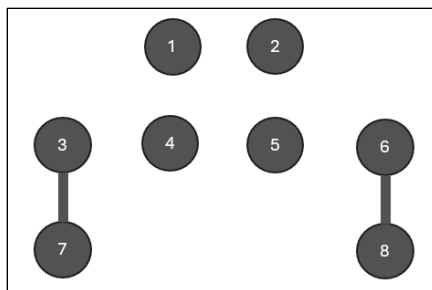


Figure 5.4. Adjust J24 Jumpers According to the Pattern for UART Mode

5. Power on the SOP.
6. Open the EdgeHUB application and verify that an image can be captured.

5.5.1.2. Capturing Normal Object Images

1. Navigate to the **Capture Tool** in EdgeHUB.
2. Choose **Normal Object** in EdgeHUB.
3. Place the normal object in the center of the test pattern.
4. Start capture.
5. Save the captured image.
6. Repeat 100 times (or 30 times for simple defect) with different normal objects.
Note: If there are fewer than 100 objects, you can reuse the same objects by rotating them randomly.

5.5.1.3. Capturing Defective Object Images

Repeat the same steps in the [Capturing Normal Object Images](#) section, but select **Abnormal Object** in EdgeHUB.

5.5.1.4. Switching from UART to I2C Mode (Optional)

1. If UART was used during data collection, switch back to I2C by updating the `/opt/hub/config/gard_12345.json` file with the following settings:
 - `"control_bus": "HUB_GARD_BUS_I2C"`
 - `"data_bus": "HUB_GARD_BUS_I2C"`
2. Power off the SOP.
3. Adjust the J24 jumpers on the middle of the carrier board according to the pattern below:

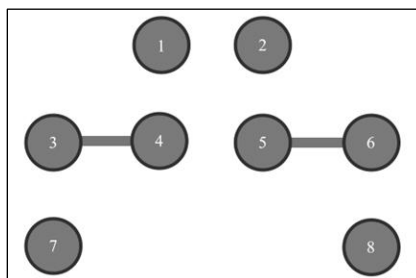


Figure 5.5. Adjust J24 Jumpers According to the Pattern for I2C Mode

4. Power on the SOP.
5. Open the EdgeHUB application and verify that an image can be captured.

5.6. Training Model

With the gathered images, run the Workbench application and follow the instructions.

[Submit a request](#) to access the Workbench application if you do not already have access.

The following figure shows the Workbench application after the model is trained and tested.

Note the **Threshold** value, as it will be used in the next step.

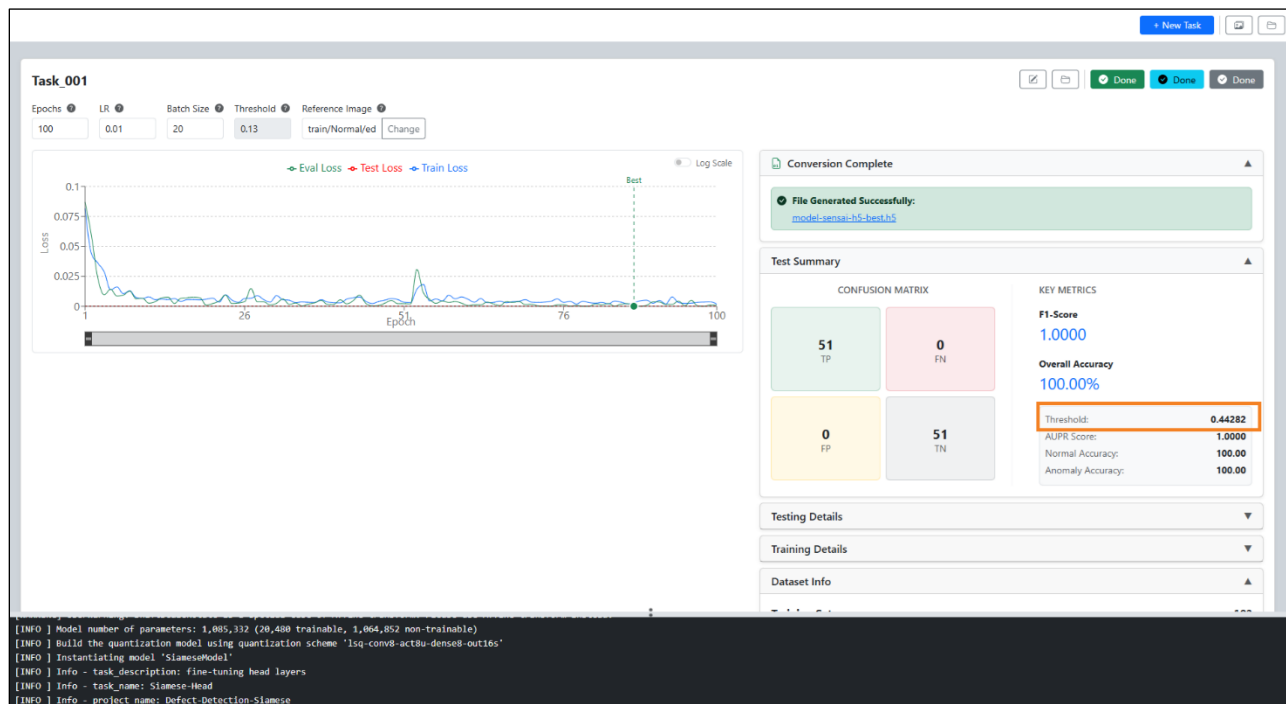


Figure 5.6. Threshold Value

5.7. Compiling and Programming for FPGA

5.7.1. Compile RISC-V Binary

Download the `defect_detection_pipeline_{version}.bin` file located in the *FIRMWARE* folder of the [sensAI Defect Detection Demo Package](#) repository.

5.7.2. Compile Defect Detection Network

1. Follow the *Compiling AI Model for FPGA* procedure in the [sensAI GARD Application Reference Design User Guide \(FPGA-RD-02332\)](#) to compile the network.
2. Alternatively, use existing networks from the *MODELS* folder of the [sensAI Defect Detection Demo Package](#) repository:
 - Shapes
 - Pills

5.7.3. Prepare Binary File with Reference Vector and Threshold

Generate the file by using the following steps:

1. Create a python environment and install the latest [ML engine simulator](#) (version 5.3 or later).
2. Download the `generate_embedding_vector_short.py` file located in the *ROOT_IMAGE* folder of the [sensAI Defect Detection Demo Package](#) repository.

3. Open the Python script in a text editor and update the following parameters:
 - `MODEL_FILE_PATH`: Keras model path
 - `REF_IMAGE_PATH`: Reference .jpeg or .png image path (any normal image from the dataset)
 - `THRESHOLD`: Set threshold value (float)
 - `OUTPUT_DIR`: Path where you want to generate the embedded vector bin file
 - `EMBEDDING_BIN_FILE`: Set name of the embedded vector output bin file
4. Execute the script to generate an embedded bin file in `OUTPUT_DIR`.
5. Alternatively, use the existing binary files from the *Shapes* demo.

5.7.4. Prepare Binary Image

Modify and execute the following command (from the [sensAI Reference Design GitHub](#) folder) to generate binary images. This command generates the *Shapes* demo binary image:

```
python .\platform_fw\src\scripts\RootImageBuilder.py -n -o fwroot.bin --mlfile
0x2001="..\..\demo\cpnx\dd\MODELS\shapes\dd-shapes-2026-01-15.bin" --camconf
0x3001="..\..\demo\cpnx\dd\CAMERA_CONFIG\Sony-IMX219-4K-Camera-Configuration-Custom.bin" --gard
0x1001="..\..\demo\cpnx\dd\FIRMWARE\defect_detection_pipeline_1.0.991a93b.bin" --rfsconf
LayoutVersion=1,UpdateCount=4,ControlField=Config-Valid,ControlField=crc-not-
valid,StartOfDirectory=0x10000,UIDOfFirmwareToBoot=0x1001 --erase-block-alignment 0x8000 --appdata
0x5001="..\..\demo\cpnx\dd\MODELS\shapes\defect_detection_ok_shapes_embedding_vector_64_bytes.bin"
```

Parameters to set with your generated binaries:

- `--mlfile`: Compiled binary file generated with the sensAI Compiler in [Compile Defect Detection Network](#).
- `--camconf`: Binary file for the Sony IMX219 configuration generated by *CameraConfigBuilder.py*.
- `--gard`: RISC-V binary from the [Compile RISC-V Binary](#) procedure.
- `--appdata`: Embedded vector binary file generated in the [Prepare Binary File with Reference Vector and Threshold](#) procedure.

5.7.5. Flash Prepared Binary Image

Use the Radiant Programmer tool with the settings shown in [Figure 4.7](#) to flash the prepared binary image from the [Prepare Binary Image](#) procedure (fwroot.bin) at address **0x003000000** and package **8-Pad WSON**.

5.8. Running Trained Model

Follow the procedure in the [Running the Demo](#) section to start the device, and connect to the web application to start the stream.

Place an object without defects under the camera and observe the bounding box around the feed:

- Green indicates a good match.
- Red indicates a detected defect.

An example of defect detection on a selected object of a trained model is shown in the following figures.

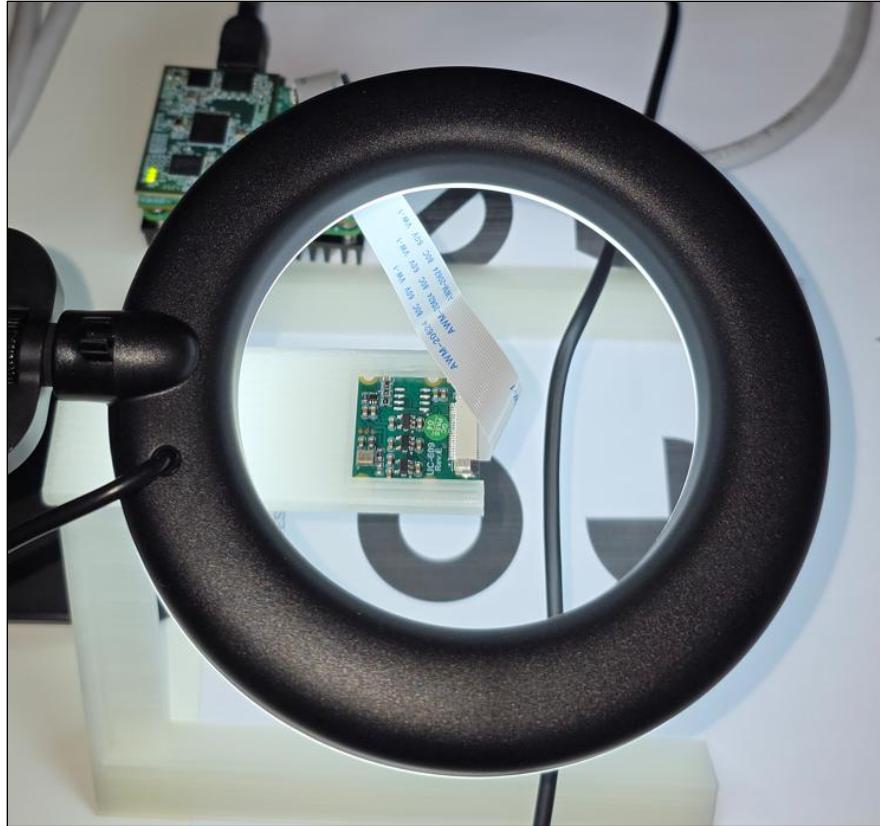


Figure 5.7. Top View of Camera with Selected Object of Trained Model



Figure 5.8. Profile View of Camera with Selected Object of Trained Model

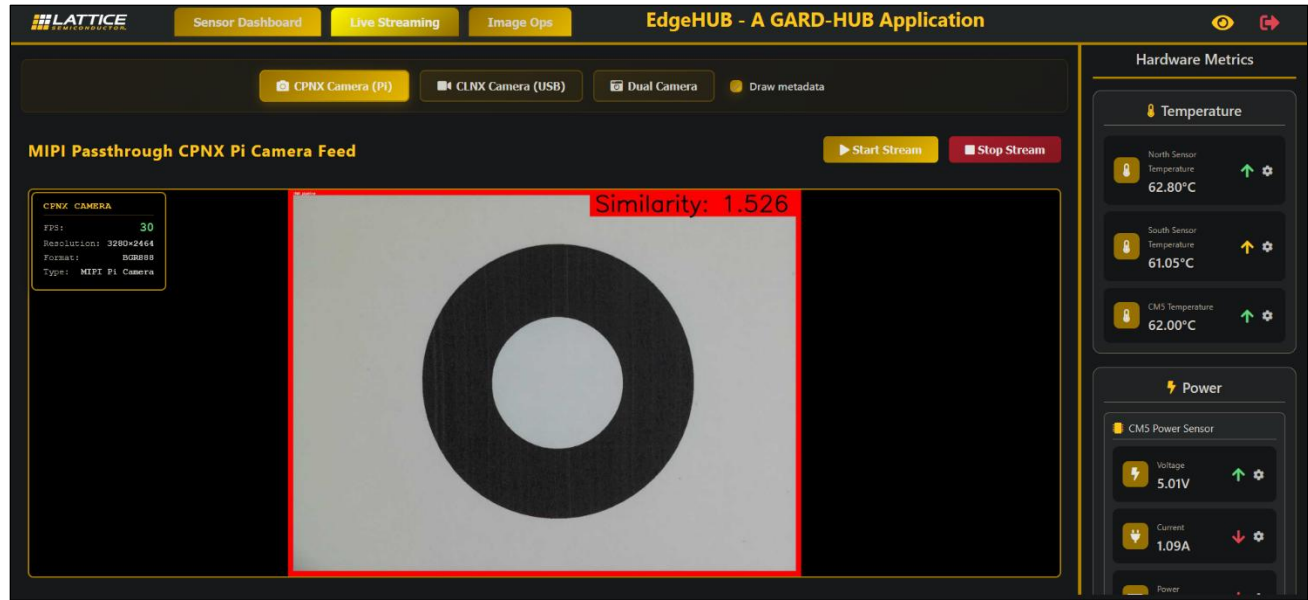


Figure 5.9. Defect Detected in Selected Object of Trained Model

References

- [sensAI GARD Application Reference Design User Guide \(FPGA-RD-02332\)](#)
- [Lattice sensAI Solution Stack](#) web page
- [sensAI Defect Detection Demo Package](#)
- [sensAI Reference Design GitHub](#)
- [CertusPro-NX](#) web page
- [MachXO3D](#) web page
- [Lattice Propel Design Environment](#) web page
- [Lattice Radiant Software](#) web page
- [Lattice Solutions IP Cores](#) web page
- [Lattice Solutions Reference Designs](#) web page
- [Lattice Insights](#) web page for Lattice Semiconductor training courses and learning plans

Technical Support Assistance

Submit a technical support case through www.latticesemi.com/techsupport.

For frequently asked questions, refer to the Lattice Answer Database at www.latticesemi.com/Support/AnswerDatabase.

Revision History

Revision 1.1, February 2026

Section	Change Summary
Running the Demo	- Updated the following sections: Compile RISC-V Binary Compile Defect Detection Network Prepare Binary Image - Updated steps 2 and 5 in the Prepare Binary File with Reference Vector and Threshold section.
References	- Updated <i>sensAI GARD Application Reference Design User Guide (FPGA-RD-02332)</i>. - Added <i>sensAI Defect Detection Demo Package</i> and <i>sensAI Reference Design GitHub</i>.

Revision 1.0, January 2026

Section	Change Summary
All	Initial release.



www.latticesemi.com